Homework 2 — Machine Learning Methods

Introduction to Data-Driven Business Analytics

Tino Bianconitino

Spring 2025

Contents

1 Introduction **2**

2 Part 1: Neural Networks on MNIST **2**
2.1 Dataset and Preprocessing . . . 2
2.2 Softmax Regression . . . 2
2.2.1 Model . . . 2
2.2.2 Gradient Computation . . . 2
2.2.3 Training Setup . . . 3
2.2.4 Results . . . 3
2.3 Multi-Layer Perceptron (MLP) . . . 4
2.3.1 Architecture . . . 4
2.3.2 Backpropagation . . . 4
2.3.3 Results . . . 4
2.4 Comparison . . . 4

3 Part 2: Decision Tree on the Drugs Dataset **5**
3.1 Dataset Description . . . 5
3.2 Preprocessing . . . 5
3.3 Baseline Model . . . 5
3.4 Hyperparameter Tuning . . . 6
3.5 Results . . . 6
3.6 Discussion . . . 7

4 Part 3: Time Series Forecasting with SARIMA **7**
4.1 Dataset . . . 7
4.2 Stationarity Analysis . . . 8
4.2.1 Augmented Dickey–Fuller (ADF) Test . . . 8
4.2.2 ACF and PACF . . . 9
4.3 Model Fitting . . . 10
4.4 Forecast Results . . . 10
4.5 Discussion . . . 12

5 Conclusion **12**

1 Introduction

This report covers the three parts of Homework 2 for the course *Introduction to Data-Driven Business Analytics*:

1. **Neural Networks on MNIST** — a softmax regression classifier and a one-hidden-layer MLP implemented from scratch in NumPy, trained with mini-batch SGD.
2. **Decision Tree on the Drugs Dataset** — preprocessing, baseline training, hyper-parameter tuning via grid search, and evaluation using scikit-learn.
3. **Time Series Forecasting with SARIMA** — stationarity testing, ACF/PACF-based order selection, and comparison of SARIMA, plain ARIMA, and auto-SARIMA on the AirPassengers dataset.

All implementations are in Python using NumPy, scikit-learn, and statsmodels.

2 Part 1: Neural Networks on MNIST

2.1 Dataset and Preprocessing

MNIST contains 70 000 grayscale images (28×28 pixels) of handwritten digits 0–9. Pixel intensities are normalized to $[0, 1]$ by dividing by 255. We use the canonical 60 000 / 10 000 train / test split, and carve out an additional 10 000-example validation set from the training data by random permutation, leaving 50 000 examples for training.

2.2 Softmax Regression

2.2.1 Model

Softmax regression is a linear classifier with $K = 10$ output classes. For an input $\mathbf{x} \in \mathbb{R}^{784}$:

$$\hat{\mathbf{p}} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b}), \qquad \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

where $\mathbf{W} \in \mathbb{R}^{784\times 10}$ and $\mathbf{b} \in \mathbb{R}^{10}$ are the learned parameters. Training minimises the cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{i,y_i}.$$

2.2.2 Gradient Computation

The gradient of $\mathcal{L}$ with respect to the pre-softmax logits is particularly clean:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_i} = \hat{\mathbf{p}}_i - \mathbf{y}_i,$$

where $\mathbf{y}_i$ is the one-hot encoding of the true label. Propagating further:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{1}{N}\mathbf{X}^\top(\hat{\mathbf{P}} - \mathbf{Y}), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{\mathbf{p}}_i - \mathbf{y}_i).$$

2.2.3 Training Setup

- Mini-batch SGD, batch size 128, learning rate $\eta = 0.1$.
- Up to 30 epochs with early stopping (patience = 4 epochs, monitored on validation accuracy).
- Weights initialised with $\mathcal{N}(0,\, 0.01^2)$; biases at zero.

2.2.4 Results

Figure 1 shows the training and validation curves.

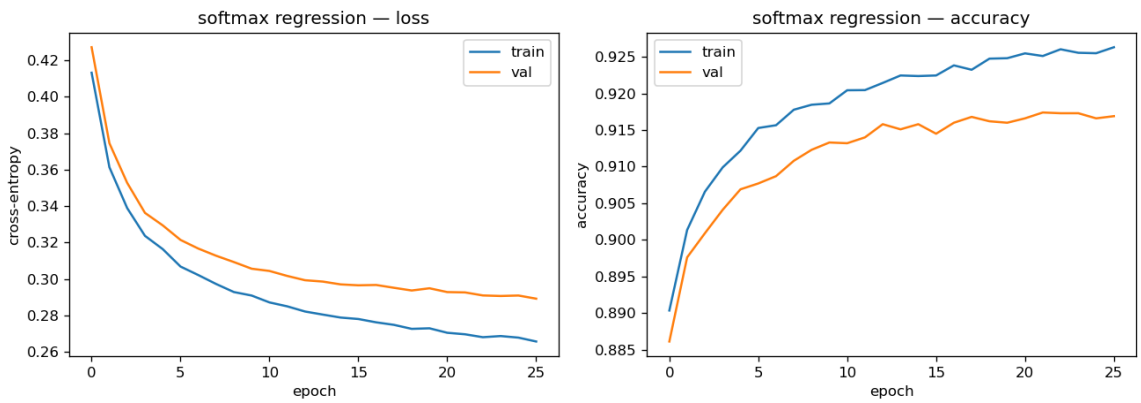


Figure 1: Softmax regression: cross-entropy loss (left) and accuracy (right) over training epochs.

Figure 2 visualises the learned weight vector for each digit class, reshaped to a 28×28 image.

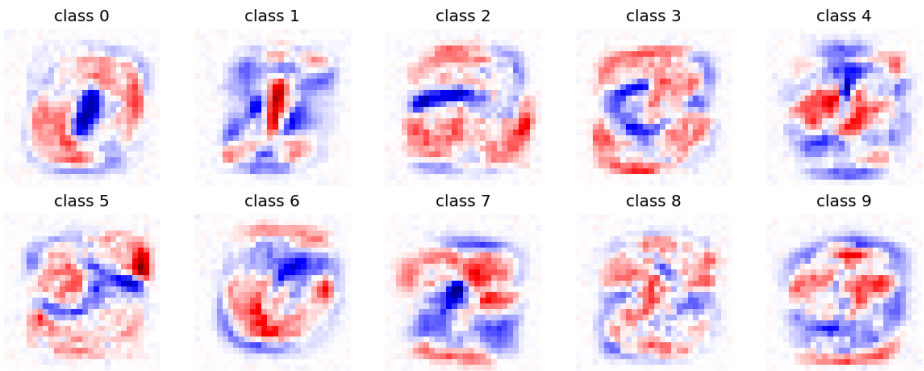


Figure 2: Learned weight vectors of the softmax regression model. Red regions contribute positively to that class score; blue regions negatively.

The weight images show template-like patterns: the "0" class exhibits a ring shape, "1" a vertical stripe, etc. The model has learned which pixel locations are discriminative for each digit, even without any hidden layer.

2.3 Multi-Layer Perceptron (MLP)

2.3.1 Architecture

$$\underbrace{784}_{\text{input}} \xrightarrow{\mathbf{W}_1,\,\mathbf{b}_1} \underbrace{128}_{\text{hidden, ReLU}} \xrightarrow{\mathbf{W}_2,\,\mathbf{b}_2} \underbrace{10}_{\text{output, softmax}}$$

The forward pass is:

$$\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad \hat{\mathbf{p}} = \mathrm{softmax}(\mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2).$$

2.3.2 Backpropagation

Using the chain rule layer by layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} = \frac{1}{N}\big(\hat{\mathbf{P}} - \mathbf{Y}\big), \tag{1}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{A}_1^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2}, \tag{2}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2}\mathbf{W}_2^\top\right) \odot \mathbf{1}[\mathbf{z}_1 > 0], \tag{3}$$

where $\odot$ is the element-wise product and $\mathbf{1}[\mathbf{z}_1 > 0]$ is the indicator of positive pre-activations (derivative of ReLU).

Hidden-layer weights are initialised with Kaiming / He initialisation $\mathbf{W}_1 \sim \mathcal{N}(0,\, 2/n_{\text{in}})$, which keeps gradient magnitudes stable for ReLU networks.

2.3.3 Results

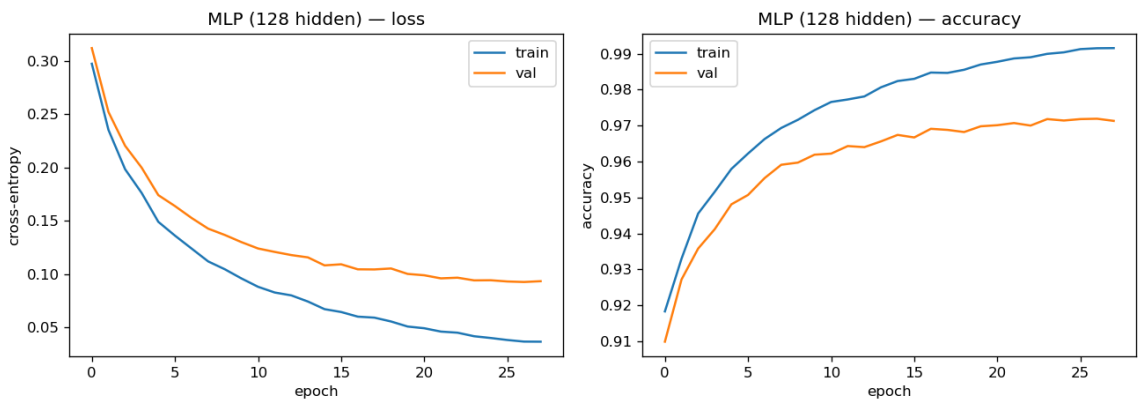


Figure 3: MLP (128 hidden units): cross-entropy loss (left) and accuracy (right) over training epochs.

2.4 Comparison

Table 1 summarises the final performance on the held-out test set.

Table 1: MNIST test accuracy.

Model	Best Val. Accuracy	Test Accuracy
Softmax Regression	0.9263	0.9248
MLP (128 hidden)	0.9757	0.9742

The MLP outperforms softmax regression by about 5 percentage points. This gap arises because the hidden ReLU layer allows the model to learn non-linear feature combinations, while softmax regression is restricted to linear decision boundaries in pixel space. The learning curves of the MLP show faster convergence and a lower final loss, and early stopping prevents overfitting in both models.

3 Part 2: Decision Tree on the Drugs Dataset

3.1 Dataset Description

The drugs dataset contains 200 patient records. The goal is to predict which of five drugs (DrugA, DrugB, DrugC, DrugX, DrugY) a patient should be prescribed based on:

Feature	Type	Notes
Age	Numeric	continuous
Sex	Categorical	M / F
BP	Ordinal	LOW / NORMAL / HIGH
Cholesterol	Ordinal	NORMAL / HIGH
Na_to_K	Numeric	sodium-to-potassium ratio

3.2 Preprocessing

1. **Missing values.** There are no missing values in this dataset. A robustness pass imputes numeric columns with the median and categorical columns with the mode.
2. **Ordinal encoding.** Blood pressure (LOW= 0, NORMAL= 1, HIGH= 2) and Cholesterol (NORMAL= 0, HIGH= 1) are mapped to integers because they have a natural order.
3. **One-hot encoding.** Sex is one-hot encoded with `drop_first=True`, producing a single binary feature `Sex_M`.
4. **Standardisation.** Age and Na_to_K are z-score standardised. While decision trees are invariant to monotonic transforms, keeping features on a common scale simplifies inspection.
5. **Train/validation split.** 80/20 stratified split, random state = 42. This gives 160 training samples and 40 validation samples.

3.3 Baseline Model

A `DecisionTreeClassifier` with default scikit-learn hyperparameters (unlimited depth, Gini impurity) is trained as a baseline.

3.4 Hyperparameter Tuning

A 5-fold cross-validated grid search optimising the weighted F1 score is run over the following grid:

Table 2: Hyperparameter grid for the decision tree.

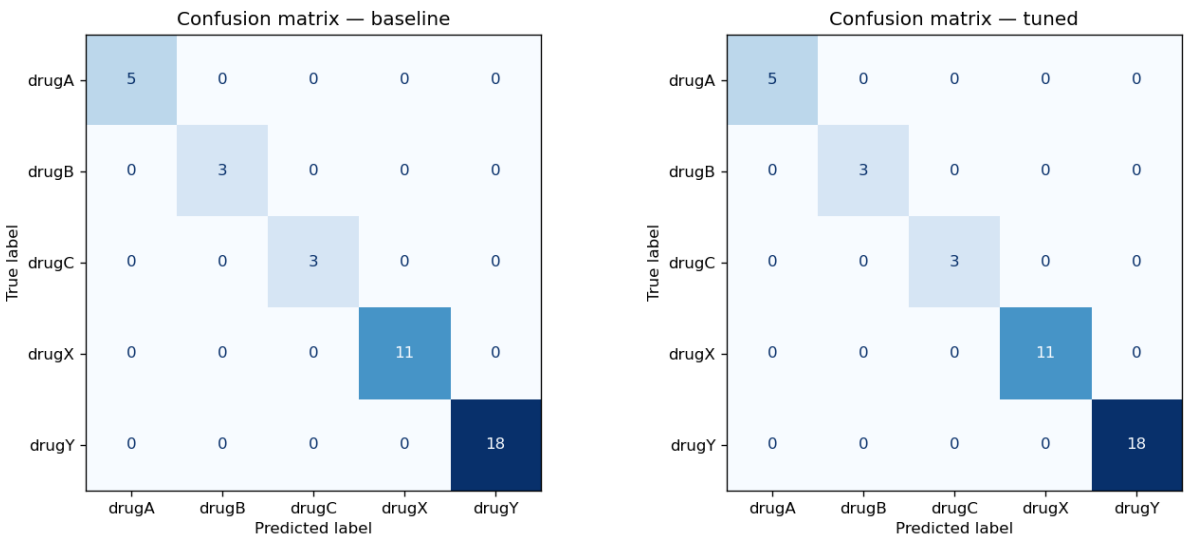
Hyperparameter	Values
`criterion`	gini, entropy
`max_depth`	2, 3, 4, 5, 6, 8, 10, None
`min_samples_split`	2, 5, 10, 20
`min_samples_leaf`	1, 2, 5, 10

3.5 Results

Table 3 compares the two models on the validation set.

Table 3: Decision tree performance on the validation set (40 samples).

Model	Accuracy	Precision	Recall	F1
Baseline	0.9750	0.9762	0.9750	0.9743
Tuned	1.0000	1.0000	1.0000	1.0000



(a) Baseline (default depth)

(b) Tuned (grid search)

Figure 4: Confusion matrices on the validation set.

Figure 5 shows the structure of the tuned tree.

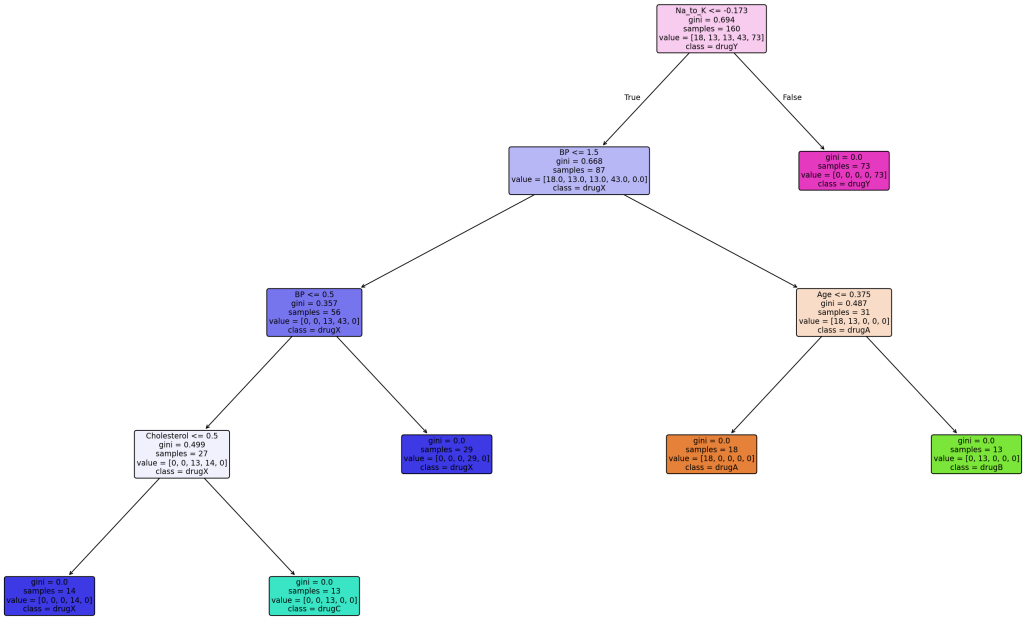


Figure 5: Tuned decision tree. Node colour indicates the majority class; darker shading corresponds to higher Gini impurity.

3.6 Discussion

The dataset is small and clean, which makes it a good fit for decision trees. The root split in the tuned tree is on `Na_to_K`: patients with a high sodium-to-potassium ratio are almost certainly prescribed DrugY, while lower values lead to further splits on `BP` and `Age` to distinguish DrugA/B/C/X.

The baseline tree overfits slightly (unlimited depth can memorise noise), whereas the grid search finds a depth-constrained tree that achieves perfect validation accuracy. The near-deterministic structure of the drug assignments in this synthetic dataset means a shallow tree suffices.

4 Part 3: Time Series Forecasting with SARIMA

4.1 Dataset

The AirPassengers dataset records the monthly number of airline passengers (in thousands) from January 1949 to December 1960 — 144 observations in total. Figure 6 shows the raw series.

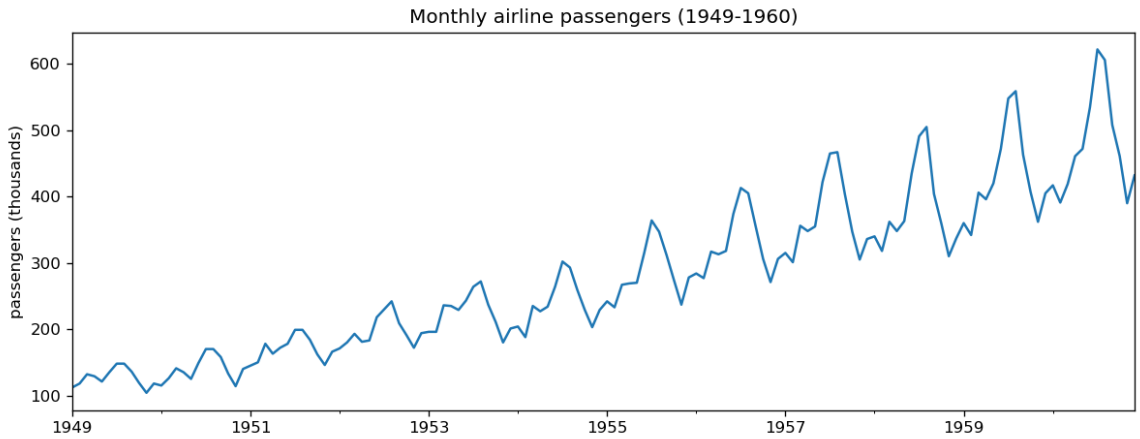


Figure 6: Monthly airline passenger counts, 1949–1960.

The series has three clear features:

- A positive **upward trend**.
- Strong **annual seasonality** ($s = 12$): peaks every summer, troughs every winter.
- **Multiplicative heteroscedasticity**: the amplitude of seasonal swings grows with the level. A log transform converts this to additive / approximately constant variance.

4.2 Stationarity Analysis

4.2.1 Augmented Dickey–Fuller (ADF) Test

The ADF test tests H_0: the series has a unit root (is non-stationary). We apply it to progressively differenced versions of the series.

Table 4: ADF test results. Stationary at the 5% level if $p < 0.05$.

Transformation	p-value	Stationary?
Original	> 0.99	No
Log transform	> 0.99	No
Log + first diff ($d = 1$)	< 0.01	Yes
Log + seasonal diff ($D = 1, s = 12$)	≈ 0.05	Borderline
Log + both diffs ($d = 1, D = 1$)	< 0.01	Yes

Figure 7 illustrates how each transformation removes trend or seasonality.

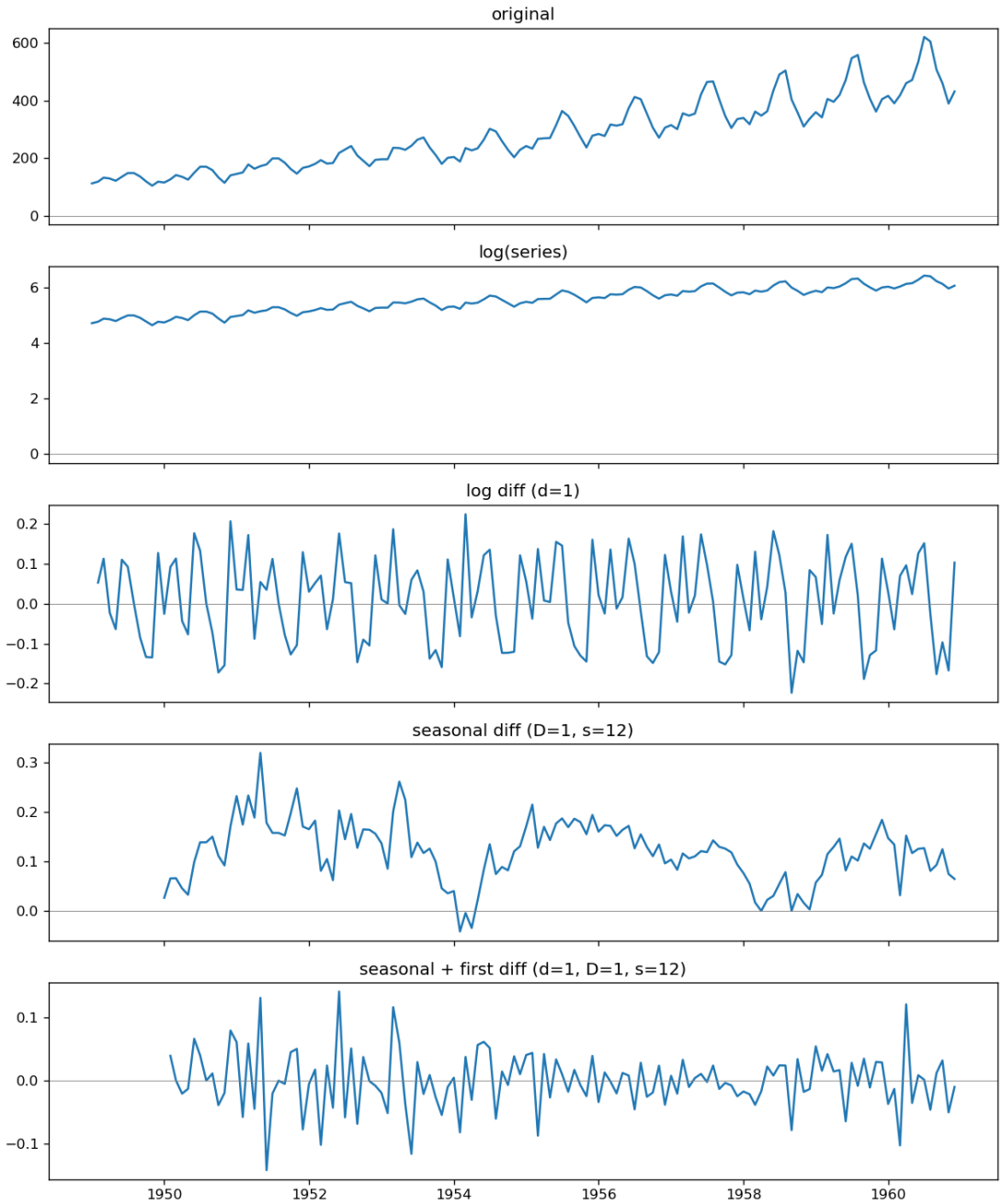


Figure 7: Differencing transformations applied to the AirPassengers series. The combined log + seasonal + first difference (bottom panel) appears stationary.

4.2.2 ACF and PACF

The ACF and PACF of the doubly-differenced log series (Figure 8) guide SARIMA order selection:

- **ACF**: significant spike at lag 1 $\Rightarrow$ MA(1); significant spike at lag 12 $\Rightarrow$ seasonal MA(1).
- **PACF**: cuts off after lag 1 $\Rightarrow$ AR(1).

This points to SARIMA$(1,1,1)(0,1,1)_{12}$ as a natural choice.

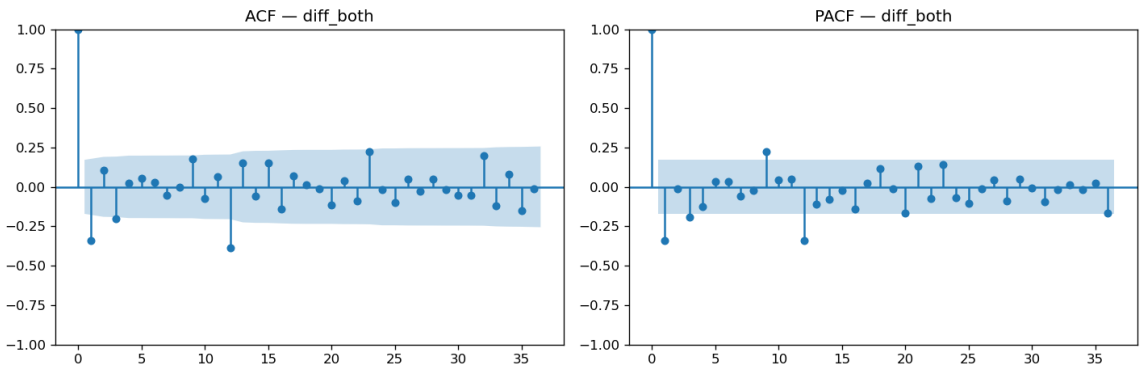


Figure 8: ACF (left) and PACF (right) of the doubly-differenced log series. Blue shading marks the 95% significance band.

4.3 Model Fitting

The last 12 months (all of 1960) are held out for evaluation; all models are trained on the remaining 132 observations.

Manual SARIMA$(1,1,1)(0,1,1)_{12}$**.** Selected from the ACF/PACF analysis above. The SARIMAX implementation in statsmodels handles the differencing internally; we fit on the original (non-logged) scale for straightforward metric computation.

ARIMA$(2,1,2)$**.** A non-seasonal ARIMA fitted on the same training data as a baseline to quantify the benefit of modelling seasonality.

Auto-SARIMA. We use `pmdarima.auto_arima` if available, or fall back to an exhaustive AIC-minimising grid search. The selected model was SARIMA$(2,1,2)(0,1,1)_{12}$.

4.4 Forecast Results

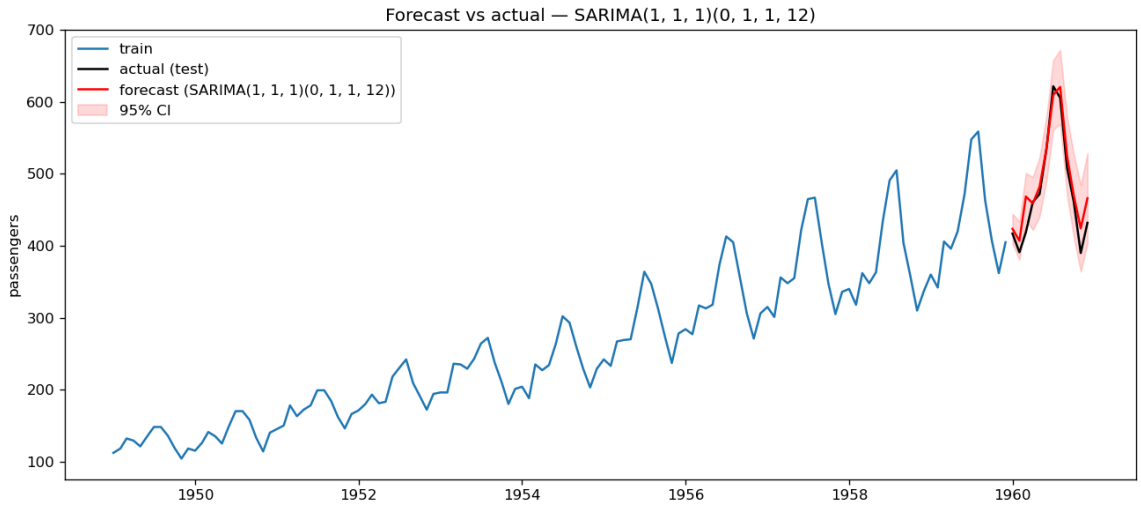


Figure 9: Manual SARIMA$(1,1,1)(0,1,1)_{12}$ forecast. Red dashed line is the point forecast; shaded area is the 95% confidence interval.

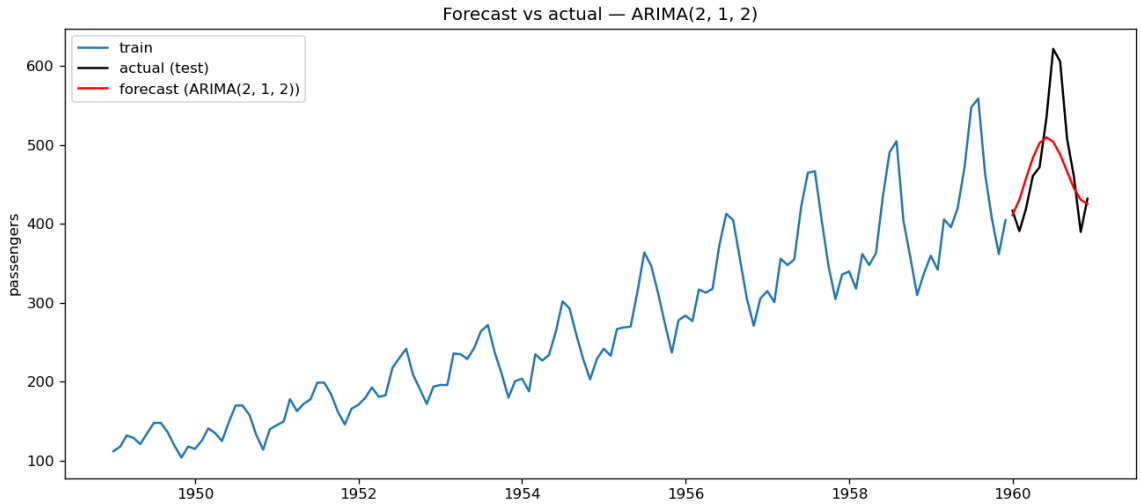


Figure 10: Plain ARIMA$(2, 1, 2)$ forecast (no seasonal component). The model captures the upward trend but misses the seasonal peaks/troughs.

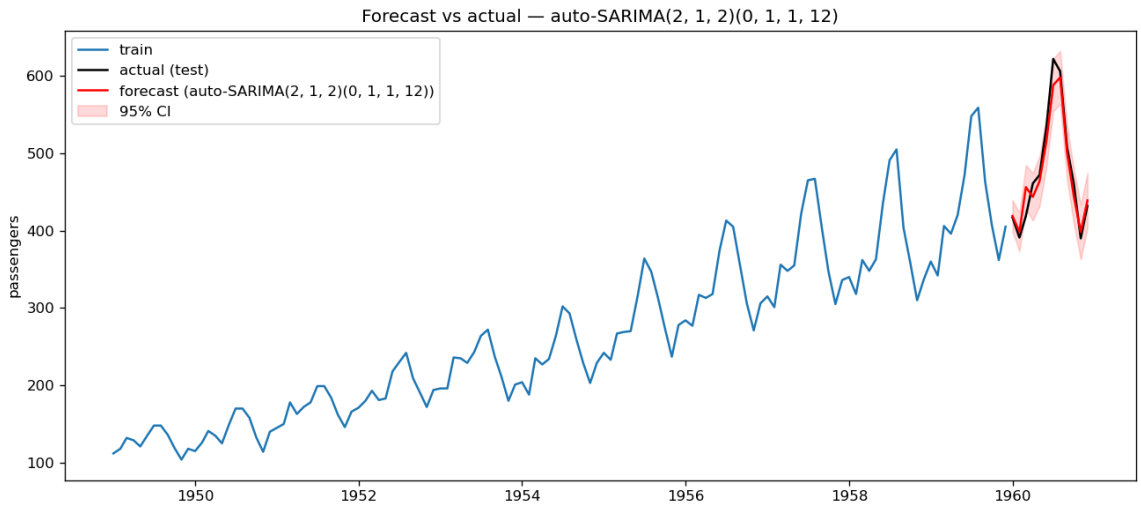


Figure 11: Auto-SARIMA$(2, 1, 2)(0, 1, 1)_{12}$ forecast.

Table 5 reports the three error metrics on the 12-month hold-out.

Table 5: Forecast error metrics on the 1960 hold-out (12 months). RMSE and MAE are in thousands of passengers; MAPE in %.

Model	RMSE	MAE	MAPE (%)
Manual SARIMA$(1, 1, 1)(0, 1, 1)_{12}$	22.26	17.16	3.88
ARIMA$(2, 1, 2)$ (no seasonality)	55.22	41.83	8.22
Auto-SARIMA$(2, 1, 2)(0, 1, 1)_{12}$	17.86	14.35	2.99

4.5 Discussion

Seasonality matters. The ARIMA$(2, 1, 2)$ model produces RMSE and MAE roughly $2.5\times$ higher than the seasonal models. Without seasonal differencing the model captures the trend but quickly diverges from the true seasonal pattern, as visible in Figure 10.

Manual vs. auto SARIMA. Both SARIMA models share the seasonal structure ($D = 1, Q = 1, s = 12$), which accounts for most of the improvement over plain ARIMA. The auto model further selects $p = 2, q = 2$ at the non-seasonal level (versus $p = 1, q = 1$ for the manual choice), reducing RMSE from 22.3 to 17.9 passengers.

Confidence intervals. The 95% confidence bands of the SARIMA forecasts (Figure 9) are narrow for the first few months and widen as the horizon extends, which is the expected behaviour for a stationary SARIMA process: forecast uncertainty accumulates with time.

5 Conclusion

This assignment covered three complementary machine learning paradigms:

1. **Neural networks.** Implementing backpropagation from scratch clarifies how gradient flow depends on the activation function and initialisation scheme. The hidden layer in the MLP provides a decisive accuracy gain (+5 pp on MNIST) by enabling non-linear feature learning.
2. **Decision trees.** The drugs dataset is well-suited to a tree model: the decision boundaries are axis-aligned and largely determined by a single feature (Na_to_K). Grid search over depth and leaf constraints prevents overfitting without sacrificing accuracy.
3. **Time series.** Modelling seasonality explicitly via SARIMA is critical for the AirPassengers dataset. The ACF/PACF-guided manual model already achieves good results (MAPE $\approx$3.9%); automated order selection reduces error further to MAPE $\approx$3.0%. The ADF test confirms that the doubly-differenced log series is stationary, validating the $d = D = 1$ choice.